



Bsc (Hons) in Computer Systems Engineering

IE3064 Embedded Systems Design

Year 3 Semester 1

Group Assignment

Line Following Arduino Car

Mohomed Nasif – IT23736900
W A Elisha Y Perera – IT23690370

Submitted Date : 2026.04.25

Table of Contents

Abstract.....	3
1. Introduction.....	3
2. Project Aim	4
3. Project Objectives	5
4. Assignment Requirement Mapping.....	5
5. System Overview	6
6. Hardware Components Used	7
7. Component Description and Justification.....	7
7.1 Arduino Uno / ATmega328P	7
7.2 TCRT5000 IR Sensors	7
7.3 VS1838 IR Receiver	8
7.4 TB6612 Motor Driver	8
7.5 N20 DC Geared Motors Without Encoders	9
7.6 Rechargeable Batteries and Buck Converter	9
7.7 Dot Board, Switch, and Jumper Wires.....	9
8. Pin Configuration.....	10
9. Hardware Block Diagram	10
10. Sensor Pattern Logic	12
11. Software Design and Control Algorithm	13
12. Overall Flowchart	14
13. Finite State Machine Design.....	14
13.1 State 1: Normal Line-Following Mode.....	15
13.2 State 2: Slow Parking-Detection Mode.....	15
13.3 State 3: Parking Execution Mode	15
14. Mode Transition Logic.....	15
15. Cross-Line Marker Detection	16
16. Duplicate Marker Prevention.....	16
17. IR Parking Signal Detection	17
18. Line-Lost Safety Function	17
19. Motor Control Logic.....	17
20. Register Level Implementation Using AVR Assembly.....	18
21. Power Distribution.....	19
22. Parking Sequence Explanation	19
23. Testing and Results	19
23.1 Sensor Testing	20
23.2 Motor Driver Testing	20
23.3 Line-Following Testing.....	20
23.4 Mode Change Testing	20
23.5 Parking Testing	20
24. Discussion	21
25. Limitations	21
26. Conclusion	22
References.....	23

Abstract

This project presents the design and implementation of an autonomous line-following and parking robot using an Arduino Uno based on the ATmega328P microcontroller. The main purpose of the robot is to follow a black line, detect special perpendicular cross-line markers, change its operating mode according to the detected markers, identify a valid parking signal, and finally perform an automatic parking sequence.

The robot uses three TCRT5000 infrared sensors to detect the black line and identify perpendicular cross-line markers. The line-following system allows the robot to move straight, correct left, correct right, and recover from small path deviations. The same three sensors are also used to detect cross-line markers when all sensors detect the black line at the same time. These markers are used to switch between normal line-following mode and slow parking-detection mode.

The robot operates using a finite state machine with three main modes: normal line-following mode, slow parking-detection mode, and parking execution mode. At startup, the robot begins in normal mode and follows the line at normal speed. When the first confirmed perpendicular cross-line is detected, the robot enters slow mode. In this mode, the robot reduces its speed and activates the IR receiver to search for a parking signal. If a valid IR signal is received, the robot enters parking mode and performs a fixed parking sequence. If no valid IR signal is received and the robot reaches the exit cross-line, it returns to normal line-following mode.

The robot is powered using two rechargeable 3.7V batteries and uses a DC-DC buck converter to provide a stable supply voltage. Two 6V N20 DC geared motors without encoders are used for movement, and a TB6612 motor driver controls motor speed and direction. Motor speed is controlled using PWM signals, while motor direction is controlled using digital output pins.

The full control logic is implemented using AVR Assembly language. This allows direct register-level control of the ATmega328P microcontroller, including input/output pin configuration, sensor reading, motor control, PWM generation, counters, flags, and timing delays. Safety and reliability features such as cross-line confirmation, duplicate marker prevention, IR signal validation, and line-lost timeout stopping are also included. Overall, this project demonstrates important embedded system concepts such as sensor interfacing, motor control, state-machine logic, low-level programming, and autonomous robotic decision-making.

1. Introduction

Autonomous robots are widely used in modern engineering applications such as industrial automation, warehouse transport systems, smart manufacturing, delivery robots, and educational robotics. One of the most common types of autonomous mobile robots is the line-following robot. A line-following robot is designed to move along a predefined path by detecting a line on the ground using sensors. Although a basic line-following robot only follows a path, this project extends that concept by adding automatic mode selection and parking functionality.

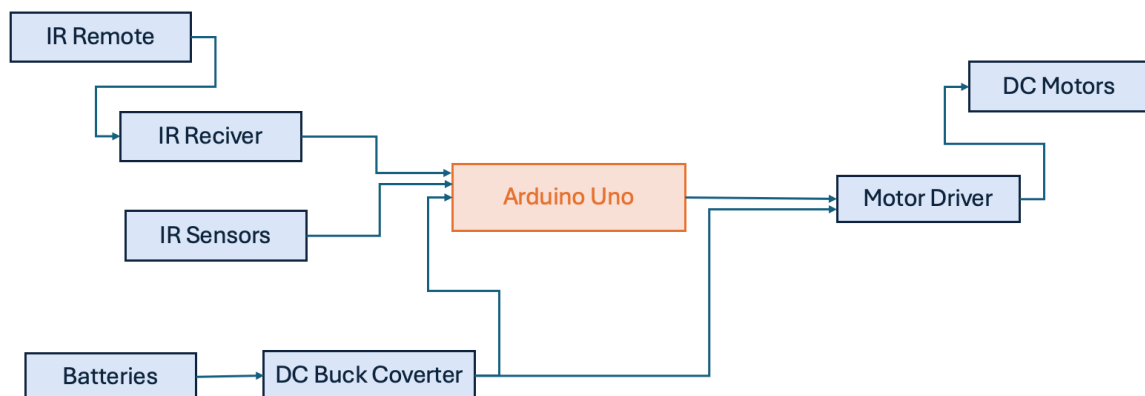
The robot developed in this project is required to follow a black line, detect perpendicular cross-line markers, slow down near a parking area, identify a selected parking bay using an IR signal, and perform an automatic parking sequence. This makes the system more advanced than a simple line follower because the robot must make decisions based on sensor input, counter values, timing, and operating modes.

The robot is controlled by an Arduino Uno board, which contains an ATmega328P microcontroller. However, instead of using normal Arduino C/C++ functions, the firmware is implemented using AVR Assembly language. This is an important part of the project because it demonstrates low-level control of the microcontroller. Through Assembly programming, the system directly accesses registers, configures input and output ports, controls timers, generates PWM signals, and manages movement logic.

Three TCRT5000 IR sensors are used for line detection. These sensors are placed at the front of the robot as left, middle, and right sensors. Based on the sensor pattern, the robot decides whether to move forward, turn left, turn right, or search for the line. A VS1838 IR receiver is used to detect the parking signal. The IR receiver is only checked when the robot is in slow parking-detection mode, which prevents the robot from parking in the wrong location.

The robot uses two N20 DC geared motors for movement. Since these motors do not include encoders, the system uses open-loop motor control. The robot cannot measure exact distance or wheel rotation, so the parking action is performed using fixed motor speeds and time delays. The motors are controlled through a TB6612 motor driver, which allows speed control using PWM and direction control using digital signals.

This project combines hardware design, embedded programming, sensor-based decision-making, and robotic movement control. It provides practical experience in designing an autonomous system that can sense its environment, change behavior based on conditions, and complete a required task without human control.



2. Project Aim

The main aim of this project is to design and implement an autonomous line-following and parking robot using Arduino Uno and AVR Assembly language. The robot should be able to follow a black line, detect perpendicular cross-line markers, change its operating mode, identify a parking signal using an IR receiver, and perform an automatic parking sequence.

3. Project Objectives

The main objectives of this project are

1. To design a robot that can follow a black line using three TCRT5000 IR sensors.
2. To implement sensor-based left, right, and straight movement correction.
3. To detect perpendicular cross-line markers using the three line sensors.
4. To use a cross-line counter to select the correct robot operating mode.
5. To change from normal line-following mode to slow parking-detection mode after detecting the first confirmed cross-line marker.
6. To continue moving slowly inside the parking detection area.
7. To activate the IR receiver only during slow parking-detection mode.
8. To confirm the IR signal before entering parking mode.
9. To execute a fixed automatic parking sequence after receiving a valid IR signal.
10. To return to normal line-following mode if the robot leaves the parking area without receiving a valid IR signal.
11. To stop the robot safely if the line is lost for a long time.
12. To implement the full control logic using AVR Assembly language without depending on Arduino libraries.

4. Assignment Requirement Mapping

This section explains how the developed robot satisfies the main assignment requirements.

Assignment Requirement	How It Was Achieved in the Project
Use Arduino Uno / ATmega328P	Arduino Uno was used as the main controller of the robot.
Use AVR Assembly programming	The control logic was implemented using AVR Assembly with direct register-level control.
Follow a black line	Three TCRT5000 IR sensors detect the line and guide the robot movement.
Detect perpendicular double/cross lines	When all three sensors detect black at the same time, the pattern becomes 111, which is treated as a cross-line marker.
Change modes using cross-line detection	A cross-line counter is used to switch between normal mode and slow parking-detection mode.
Slow down near the parking area	The first confirmed cross-line changes the robot from normal speed to slow speed.
Search for parking signal only in parking area	The IR receiver is checked only when the robot is in slow mode.
Identify selected parking bay	A valid IR signal received during slow mode confirms the selected parking bay.
Perform parking movement	The robot performs a timed sequence: forward, stop, pivot left, reverse, and final stop.
Avoid false marker counting	A marker latch flag prevents the same cross-line from being counted multiple times.
Avoid false IR activation	An IR confirmation counter is used before entering parking mode.
Stop safely if line is lost	If all three sensors detect no line for a defined timeout, the robot stops for safety.

This mapping shows that the project is fully aligned with the expected assignment logic and demonstrates all required robot functions.

5. System Overview

The robot works as a finite state machine. A finite state machine is a control method where a system behaves differently depending on its current state or mode. This method is suitable for this project because the robot must perform different actions during line following, parking detection, and parking execution.

The robot has three main operating modes

Mode	Name	Function
MODE_NORMAL	Normal Line-Following Mode	Robot follows the black line at normal speed.
MODE_SLOW	Slow Parking-Detection Mode	Robot slows down and checks for the parking IR signal.
MODE_PARK	Parking Execution Mode	Robot performs the automatic parking sequence and stops.

At startup, the robot begins in MODE_NORMAL. In this mode, it follows the black line using the three IR sensors. The IR receiver is ignored in normal mode because parking should not happen outside the parking detection area.

When the robot detects the first confirmed perpendicular cross-line marker, it changes to MODE_SLOW. In this mode, the robot continues following the line but at a lower speed. This slow movement gives the robot more time to detect the parking signal accurately. The IR receiver becomes active only in this mode.

If the IR receiver detects a valid signal while the robot is in slow mode, the robot changes to MODE_PARK. In parking mode, the robot stops following the line and executes a fixed parking movement sequence. After parking is completed, the robot stops permanently.

If no valid IR signal is detected while the robot is moving through the parking area, the robot continues in slow mode until it reaches the exit cross-line. After detecting the final cross-line, the robot returns to normal line-following mode and continues along the path.

6. Hardware Components Used

Component	Quantity	Purpose
Arduino Uno with ATmega328P	1	Main microcontroller for controlling the robot
N20 DC geared motors, 6V, without encoders	2	Drive the left and right wheels
TB6612 motor driver	1	Controls motor speed and direction
DC-DC buck converter	1	Regulates battery voltage to a stable supply
Rechargeable batteries, 3.7V, 5000mAh	2	Main power source
VS1838 IR receiver	1	Detects parking signal
TCRT5000 IR sensors	3	Detect black line and cross-line markers
Dot board	1	Provides stable wiring and connections
Switch	1	Turns the robot power on and off
Jumper wires	As required	Used for circuit connections

7. Component Description and Justification

7.1 Arduino Uno / ATmega328P

The Arduino Uno is used as the main control unit of the robot. It contains the ATmega328P microcontroller, which reads input signals from the IR sensors and IR receiver. It also sends output signals to the TB6612 motor driver to control the two motors.

The ATmega328P is suitable for this project because it has enough input and output pins for the three line sensors, IR receiver, motor driver direction pins, PWM pins, and standby control. It also includes hardware timers that can be used to generate PWM signals for motor speed control.

In this project, the Arduino Uno is not programmed using normal Arduino functions such as `digitalWrite()`, `analogWrite()`, or `delay()`. Instead, the robot is programmed using AVR Assembly language. This gives direct access to the microcontroller hardware. Assembly programming allows the program to configure registers, read input pins, write output pins, control timers, and manage program flow at a low level.

Using AVR Assembly improves the educational value of the project because it shows how embedded systems work internally. It also provides better understanding of input/output control, bit manipulation, timer configuration, PWM generation, and real-time decision-making.

7.2 TCRT5000 IR Sensors

Three TCRT5000 IR sensors are used to detect the black line. Each sensor contains an infrared transmitter and receiver. The transmitter emits infrared light toward the surface, and the receiver measures the reflected light.

A white or light-colored surface reflects more infrared light, while a black surface absorbs more infrared light. Because of this difference, the sensor can identify whether it is above the black line or the background surface.

The sensors are placed at the front of the robot as:

Sensor	Arduino Pin	Function
Left sensor	A0	Detects line position on the left side
Middle sensor	A1	Detects center line position
Right sensor	A2	Detects line position on the right side

The middle sensor is mainly used to keep the robot moving straight. The left and right sensors are used to detect deviations from the line. If the line moves toward the left sensor, the robot turns left. If the line moves toward the right sensor, the robot turns right.

The same sensors are also used to detect perpendicular cross-line markers. When all three sensors detect the black line at the same time, the sensor pattern becomes 111. This indicates that the robot is passing over a cross-line marker. These markers are used to change the robot mode.

7.3 VS1838 IR Receiver

The VS1838 IR receiver is used to detect the parking signal. It is connected to Arduino digital pin D3. The purpose of this receiver is to identify whether the robot has reached the correct parking bay.

The IR receiver is not checked during normal line-following mode. It is checked only when the robot is in slow parking detection mode. This is an important design decision because it prevents the robot from responding to IR signals outside the parking area.

The IR receiver output usually becomes LOW when IR activity is detected. However, a single signal detection is not enough to enter parking mode. To improve reliability, the program uses an IR confirmation counter. The IR signal must be detected repeatedly until the counter reaches a defined threshold. Only then does the robot enter parking mode.

This prevents false parking caused by noise, random IR signals, or temporary signal disturbances.

7.4 TB6612 Motor Driver

The TB6612 motor driver is used to control the two N20 DC motors. The Arduino Uno cannot directly supply enough current to drive the motors, so a motor driver is required.

The TB6612 allows the robot to control both speed and direction. Motor direction is controlled using digital signals, while speed is controlled using PWM signals.

The motor driver has the following main control pins:

Pin	Function
AIN1, AIN2	Control direction of Motor A

BIN1, BIN2	Control direction of Motor B
PWMA	Speed control for Motor A
PWMB	Speed control for Motor B
STBY	Enables or disables the motor driver

The TB6612 is more efficient than older motor drivers such as the L298N. It produces less heat and provides smoother motor control. This makes it suitable for a small battery-powered robot.

7.5 N20 DC Geared Motors Without Encoders

Two 6V N20 DC geared motors are used to move the robot. One motor drives the left wheel, and the other motor drives the right wheel. By controlling the speed and direction of both motors, the robot can move forward, turn left, turn right, pivot, reverse, and stop.

The motors used in this project do not have encoders. This means the robot cannot measure exact wheel rotation, speed, or distance travelled. Because of this, the robot uses open loop control. The parking sequence is controlled using fixed PWM speeds and time delays.

Although motors with encoders would provide more accurate control, the N20 motors without encoders are suitable for this project because the robot follows a fixed path and performs a predefined parking sequence.

7.6 Rechargeable Batteries and Buck Converter

The robot is powered using two rechargeable 3.7V batteries rated at 5000mAh. These batteries provide the main power for the robot.

A DC-DC buck converter is used to regulate the battery voltage and provide a stable 5V supply for the Arduino, sensors, IR receiver, and motor driver logic. Stable voltage is important because unstable power can affect sensor readings, microcontroller operation, and motor behavior.

The motor driver receives motor supply from the battery side and logic supply from the regulated 5V side. A common ground is connected between the Arduino, motor driver, sensors, IR receiver, and battery system. This common ground is necessary because all signals must share the same voltage reference.

7.7 Dot Board, Switch, and Jumper Wires

A dot board is used to organize and stabilize the wiring. A moving robot can suffer from loose wire connections if only a breadboard is used. Therefore, using a dot board improves reliability during testing and movement.

A switch is added to turn the robot on and off easily. Jumper wires are used to connect the sensors, motor driver, Arduino, and power system.

Good wire management is important because loose or messy wiring can cause sensor errors, motor failures, or sudden resets.

8. Pin Configuration

Component	Signal / Function	Arduino Pin	Type
Left TCRT5000 sensor	Left line detection	A0	Input
Middle TCRT5000 sensor	Center line detection	A1	Input
Right TCRT5000 sensor	Right line detection	A2	Input
VS1838 IR receiver	Parking signal input	D3	Input
TB6612 AIN1	Motor A direction	D2	Output
TB6612 AIN2	Motor A direction	D4	Output
TB6612 PWMA	Motor A speed	D5 PWM	Output
TB6612 PWMB	Motor B speed	D6 PWM	Output
TB6612 BIN1	Motor B direction	D7	Output
TB6612 BIN2	Motor B direction	D8	Output
TB6612 STBY	Motor driver enable	D9	Output

9. Hardware Block Diagram

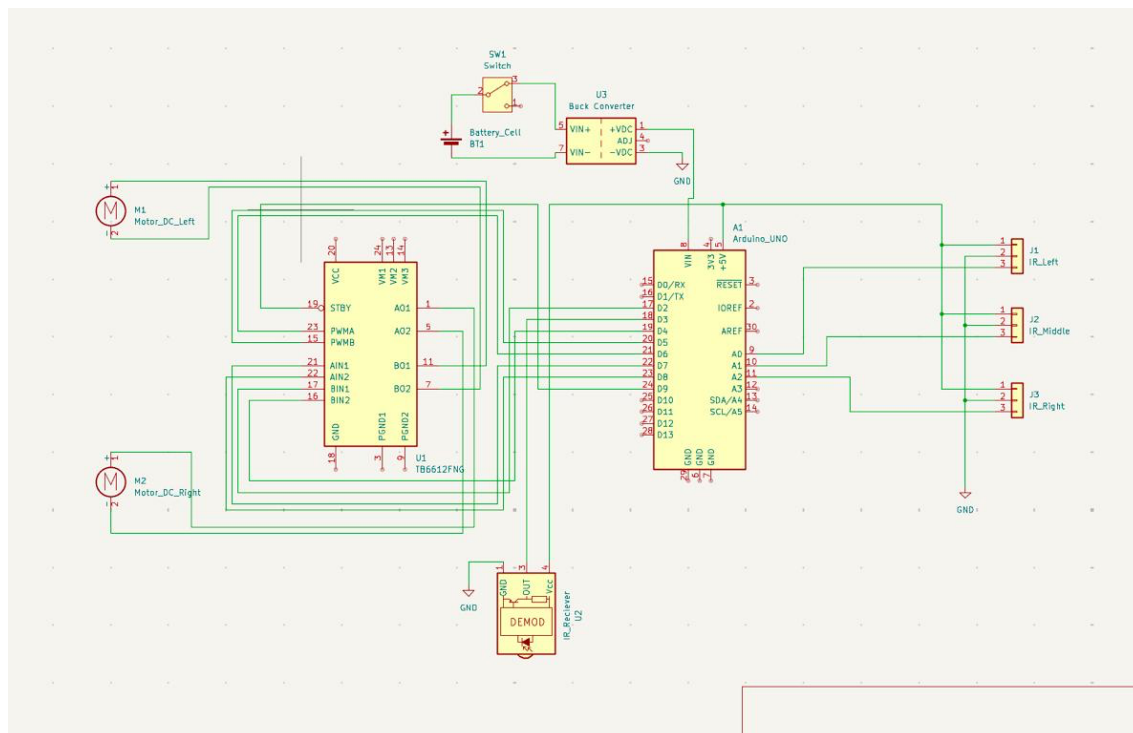
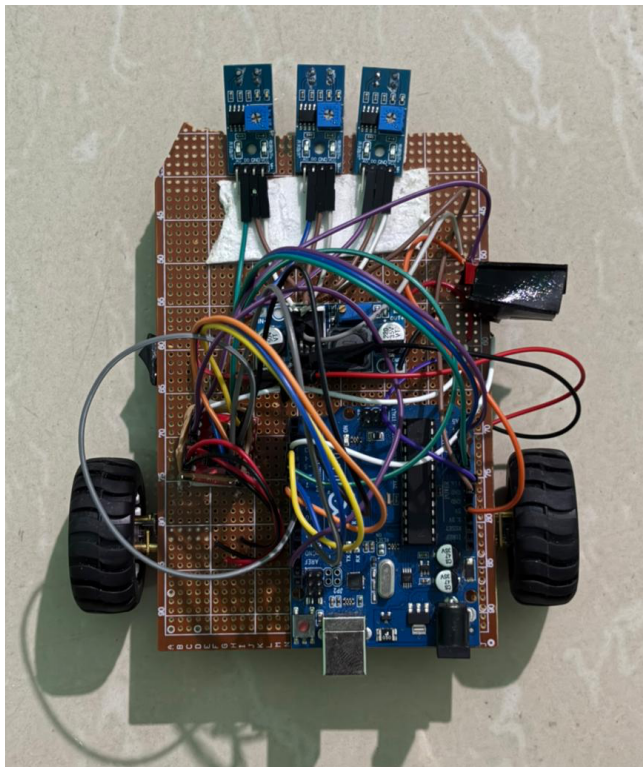
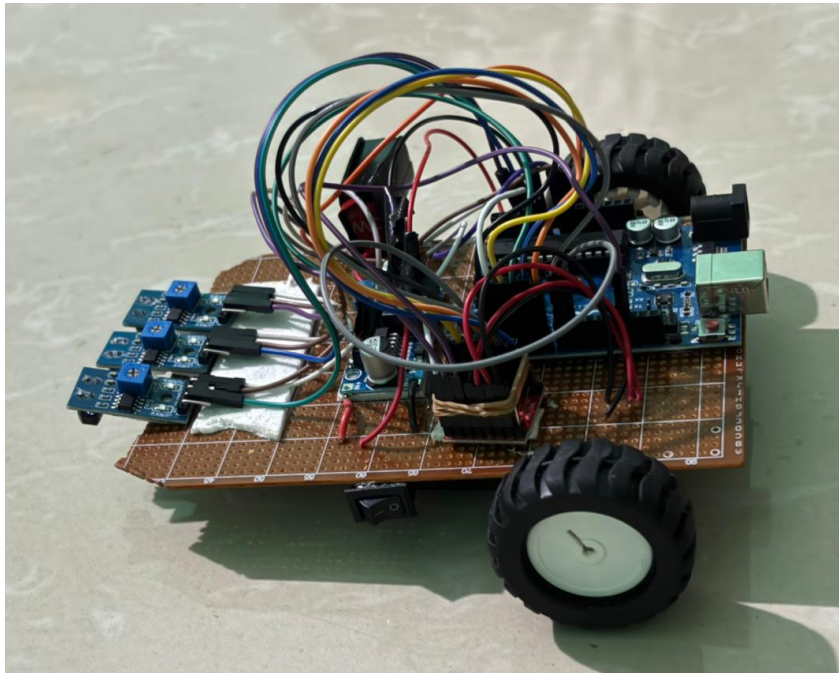
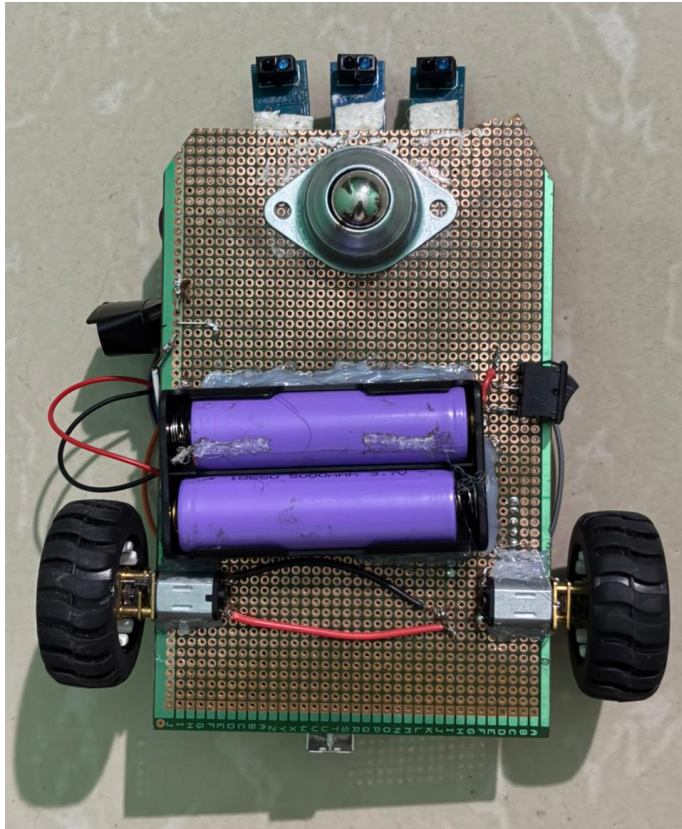


Figure 1. Hardware Block Diagram of the Autonomous Line Following and Parking Robot

Figure 1 shows the overall hardware structure of the robot. The Arduino Uno acts as the main controller and receives input from the three TCRT5000 line sensors and the VS1838 IR receiver. Based on the sensor readings and operating mode, the Arduino sends direction and PWM signals to the TB6612 motor driver. The motor driver controls the two N20 DC geared motors. The battery pack provides power to the motor system, while the buck converter provides a stable regulated voltage for the controller and sensors.





10. Sensor Pattern Logic

The three IR sensors are converted into a 3-bit sensor pattern. Each bit represents one sensor.

Bit	Sensor
Bit 2	Left sensor
Bit 1	Middle sensor
Bit 0	Right sensor

The sensor pattern is used to decide the movement of the robot.

Sensor Pattern	Meaning	Robot Action
010	Middle sensor detects line	Move straight
110	Left and middle sensors detect line	Soft left correction
100	Only left sensor detects line	Hard left correction
011	Middle and right sensors detect line	Soft right correction
001	Only right sensor detects line	Hard right correction
111	All sensors detect black line	Cross-line marker detected
101	Left and right sensors detect line	Move forward through center gap
000	No sensor detects line	Line-lost condition

This pattern-based logic allows the robot to make simple but effective movement decisions. Instead of using complex image processing or advanced sensors, the robot can follow the path using only three digital sensor readings.

11. Software Design and Control Algorithm

The software is written using AVR Assembly language. The program directly controls the ATmega328P hardware registers instead of using Arduino libraries.

At the beginning of the program, the system performs initialization. During initialization, the stack pointer is set, the motor driver pins are configured as outputs, the sensor pins are configured as inputs, the motor driver is enabled, Timer0 is configured for PWM generation, counters and flags are cleared, and the starting mode is set to MODE_NORMAL.

After initialization, the robot enters the main loop. The main loop runs continuously while the robot is operating. In each cycle, the robot reads the three line sensors, creates the sensor pattern, checks whether the line is lost, checks whether a cross-line marker is detected, updates the current mode, checks the IR receiver if the robot is in slow mode, and performs the correct motor movement.

The main software logic can be summarized as follows:

1. Start the program.
2. Initialize ports, timers, counters, flags, and motor driver.
3. Set robot mode to MODE_NORMAL.
4. Read the three line sensors continuously.
5. Convert sensor readings into a 3-bit pattern.
6. Check whether the robot has lost the line.
7. If the pattern is 111, check for cross-line marker confirmation.
8. If a confirmed cross-line is detected, update the cross-line counter.
9. Change the robot mode according to the cross-line counter.
10. If the robot is in slow mode, check the IR receiver.
11. If a valid IR signal is confirmed, enter parking mode.
12. Move the robot according to the current sensor pattern and mode.
13. Repeat the process until parking is completed or the robot stops safely.

12. Overall Flowchart

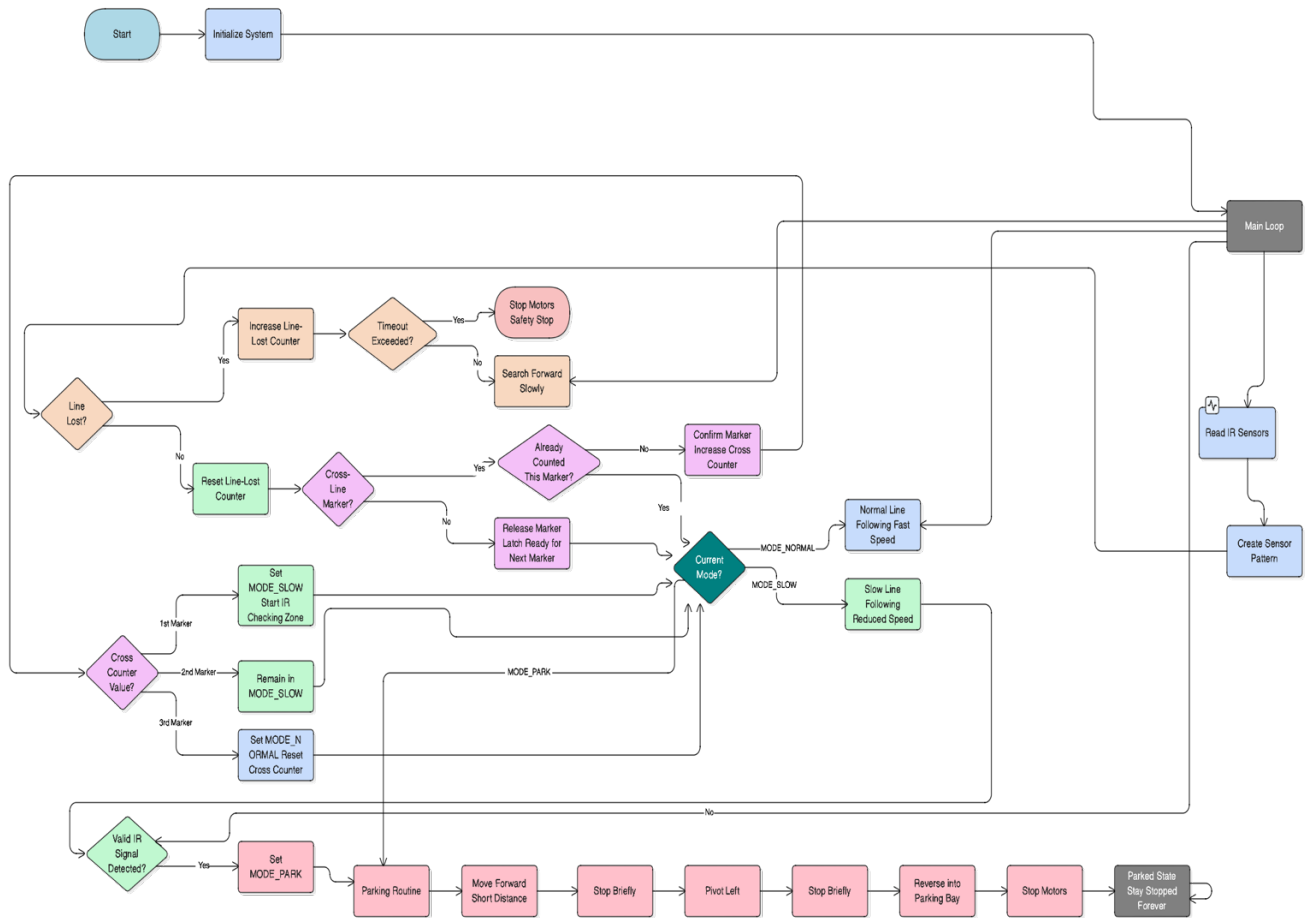


Figure 2. Overall Control Flow

Figure 2 shows the complete control flow of the robot. The program begins with system initialization and then enters a continuous main loop. In each loop cycle, the robot reads the line sensors, checks for line-lost conditions, detects cross-line markers, updates the operating mode, checks the IR receiver only during slow parking-detection mode, and performs movement according to the selected mode. This flowchart clearly shows how the robot changes between normal line-following mode, slow parking-detection mode, and parking execution mode.

13. Finite State Machine Design

The robot control system was designed as a finite state machine because the robot must behave differently depending on its current condition. This method makes the robot behavior organized, predictable, and easier to debug.

13.1 State 1: Normal Line-Following Mode

Normal mode is the default operating mode. In this mode, the robot follows the black line at normal speed. The three IR sensors continuously detect the line position, and the robot adjusts motor speeds to stay on the path.

The IR receiver is ignored in this mode because the robot should not park outside the parking detection area. This prevents accidental parking due to unwanted IR signals.

13.2 State 2: Slow Parking-Detection Mode

Slow mode begins when the robot detects the first confirmed perpendicular cross-line marker. In this mode, the robot reduces its speed but continues following the line.

The purpose of slow mode is to give the robot enough time to detect the parking signal accurately. The IR receiver becomes active only in this mode. If the robot receives a valid IR signal during slow mode, it enters parking mode.

If no IR signal is received and the robot reaches the final exit cross-line, the robot leaves slow mode and returns to normal line-following mode.

13.3 State 3: Parking Execution Mode

Parking mode starts only after a valid IR signal is detected in slow mode. Once the robot enters parking mode, it stops normal line following and performs a fixed parking sequence.

The parking sequence is:

1. Move forward slightly.
2. Stop briefly.
3. Pivot left.
4. Stop briefly.
5. Reverse into the parking bay.
6. Stop the motors.
7. Remain stopped permanently.

This parking sequence is timing-based because the robot uses motors without encoders.

14. Mode Transition Logic

The mode transition logic is controlled by the cross-line counter and the IR receiver.

Event	Current Mode	Next Mode	Explanation
Startup	None	MODE_NORMAL	Robot begins normal line following
First confirmed cross-line	MODE_NORMAL	MODE_SLOW	Robot enters parking detection area
Valid IR signal detected	MODE_SLOW	MODE_PARK	Robot starts parking sequence

Exit cross-line detected without IR signal	MODE_SLOW	MODE_NORMAL	Robot leaves parking area and continues normally
Parking sequence completed	MODE_PARK	Stopped	Robot stops permanently

This logic ensures that parking can only happen in the correct area. The robot must first detect the cross-line and enter slow mode before the IR receiver can trigger parking.

15. Cross-Line Marker Detection

Cross-line markers are used to change the operating mode of the robot. A cross-line is detected when all three line sensors detect the black line at the same time. This creates the sensor pattern **111**

However, the robot does not immediately count every 111 reading as a cross-line. This is because sensor readings can be affected by noise, vibration, lighting changes, and temporary contact with the marker. If every 111 reading was counted, the same physical marker could be counted several times while the robot is still passing over it.

To prevent this issue, the program uses a confirmation counter called `crossConfirm`. The marker must be detected for a stable period before it is accepted as a valid cross-line.

Once the marker is confirmed, the cross-line counter is increased.

Cross Counter Value	Meaning	Mode Action
1	First marker detected	Change to slow parking-detection mode
2	Robot is still inside parking detection region	Remain in slow mode
3 / final exit marker	Parking region completed	Return to normal mode and reset counter

This method allows the robot to slow down near the parking area, check for a parking signal, and return to normal if no parking signal is received.

16. Duplicate Marker Prevention

When the robot moves over a cross-line marker, the sensors may read 111 for several loop cycles. Without duplicate marker prevention, the same marker could be counted multiple times. This would cause incorrect mode changes.

To solve this problem, the program uses a marker latch flag called `FLAG_ON_MARKER`.

When a valid marker is detected and counted, the flag is set. While this flag is active, the program does not count another marker. The flag is cleared only after the robot leaves the marker and the sensor pattern is no longer 111.

A release counter is used to confirm that the robot has fully left the marker before allowing another marker to be counted.

This improves reliability because each physical cross-line is counted only once.

17. IR Parking Signal Detection

The VS1838 IR receiver is used to detect the parking signal. It is connected to digital pin D3.

The IR receiver is checked only when the robot is in `MODE_SLOW`. This is a very important part of the logic. If the IR receiver was active all the time, the robot could detect an IR signal outside the parking area and start parking at the wrong location.

The IR receiver output becomes LOW when IR activity is detected. However, one LOW reading is not enough to start parking. The program uses an IR confirmation counter. The signal must be detected repeatedly until the counter reaches a threshold.

The logic is:

1. Check whether the robot is in slow mode.
2. If not in slow mode, ignore the IR receiver.
3. If in slow mode, read the IR receiver input.
4. If IR activity is detected, increase the IR counter.
5. If the IR counter reaches the threshold, enter parking mode.
6. If the signal is not stable, do not enter parking mode.

This reduces false triggering and makes the parking decision more reliable.

18. Line-Lost Safety Function

The robot includes a safety function for line loss. If all three sensors fail to detect the line, the sensor pattern becomes **000**. This means the robot may have lost the path. However, the robot does not stop immediately because a short line gap, curve, or sensor noise could temporarily produce the 000 pattern.

Instead, the robot enters a slow searching behavior. It moves slowly while increasing a line-lost counter. If the line is detected again before the timeout, the counter resets and the robot continues normal operation.

If the line is not detected again within the defined timeout, the robot stops both motors and enters a safety stop condition. This is important because it prevents the robot from moving away from the track uncontrollably.

19. Motor Control Logic

The robot uses two DC motors ; one for the left wheel and one for the right wheel. These motors are controlled by the TB6612 motor driver.

Motor direction is controlled using digital output pins, and motor speed is controlled using PWM signals.

Movement	Left Motor	Right Motor	Result
Move straight	Forward normal speed	Forward normal speed	Robot moves forward
Soft left	Slower	Faster	Robot corrects left smoothly
Hard left	Very slow/reverse	Forward	Robot turns sharply left
Soft right	Faster	Slower	Robot corrects right smoothly
Hard right	Forward	Very slow/reverse	Robot turns sharply right
Stop	Stop	Stop	Robot stops
Reverse	Reverse	Reverse	Robot moves backward
Pivot left	One motor reverse/slow	Other motor forward	Robot rotates left

For line following, the robot does not always stop and turn sharply. Instead, it changes the relative speeds of the left and right motors. This makes movement smoother and improves path tracking.

During normal mode, the robot uses higher PWM values. During slow mode, lower PWM values are used. During parking mode, fixed PWM values and delays are used to complete the parking sequence.

20. Register Level Implementation Using AVR Assembly

The full firmware is implemented using AVR Assembly language. This means the program controls the ATmega328P microcontroller directly at register level.

The data direction registers are used to configure pins as inputs or outputs. For example, the motor driver control pins are configured as outputs because the Arduino sends signals to the motor driver. The sensor pins and IR receiver pin are configured as inputs because the Arduino reads signals from them.

The port registers are used to control output pins. These outputs control motor direction pins and the motor driver standby pin. The input pin registers are used to read the sensor values.

Timer0 is configured to generate PWM signals for motor speed control. PWM allows the robot to control motor speed by changing the duty cycle of the signal. Higher duty cycle produces higher motor speed, while lower duty cycle produces slower movement.

Counters and flags are also implemented in Assembly. These are used for cross-line confirmation, duplicate marker prevention, IR signal validation, line-lost timeout, and mode control.

Using Assembly language improves understanding of how the microcontroller works internally. It also shows how embedded systems can be controlled without high-level libraries.

21. Power Distribution

The robot is powered by two rechargeable 3.7V batteries. The battery supply is used to power the motor system through the TB6612 motor driver. A DC-DC buck converter is used to provide stable regulated voltage for the Arduino, sensors, IR receiver, and motor driver logic.

Component	Power Source
Arduino Uno	Regulated 5V from buck converter
TCRT5000 sensors	Regulated 5V
VS1838 IR receiver	Regulated 5V
TB6612 VCC	Regulated 5V
TB6612 VM	Battery supply
N20 motors	Motor driver output
Ground	Common ground for all components

A common ground is required between all parts of the circuit. Without a common ground, the Arduino may not correctly read sensor signals or control the motor driver.

Stable power distribution is important because motor current changes can cause voltage drops. If the voltage is unstable, the Arduino may reset, sensors may give incorrect readings, or motors may behave unpredictably.

22. Parking Sequence Explanation

The parking sequence starts only after the robot is in slow mode and a valid IR signal is confirmed.

Once parking mode begins, the robot stops using line-following logic. It performs a fixed sequence of movements using motor control and time delays.

The parking sequence is

1. **Move forward slightly** - The robot moves forward a short distance to align itself before turning.
2. **Stop briefly** - The robot stops to stabilize before the turning action.
3. **Pivot left** - The robot performs a left pivot turn to face the parking bay.
4. **Stop briefly again** - This pause improves stability before reversing.
5. **Reverse into the parking bay** - The robot moves backward into the parking position.
6. **Final stop** - The robot stops both motors.
7. **Permanent stop** - After parking is completed, the robot remains stopped and does not return to line following.

This method is suitable for the project because the robot uses motors without encoders. Since exact distance cannot be measured, the robot uses fixed delay-based movement.

23. Testing and Results

Testing was carried out in stages to make sure each part of the robot worked correctly before testing the full system.

23.1 Sensor Testing

Test Case	Expected Result	Observed Result	Status
Left sensor placed on black line	Left sensor detects black	Left sensor output changed correctly	Pass
Middle sensor placed on black line	Middle sensor detects black	Middle sensor output changed correctly	Pass
Right sensor placed on black line	Right sensor detects black	Right sensor output changed correctly	Pass
All sensors placed on black cross-line	Pattern becomes 111	Cross-line pattern detected	Pass
Sensors placed on white surface	Pattern becomes 000 or no line state	No-line condition detected	Pass

23.2 Motor Driver Testing

Test Case	Expected Result	Observed Result
Forward command	Both motors rotate forward	Robot moved forward
Stop command	Both motors stop	Robot stopped
Left correction	Robot turns left	Robot corrected left
Right correction	Robot turns right	Robot corrected right
Reverse command	Both motors rotate backward	Robot reversed
Pivot command	Robot rotates in place	Robot pivoted successfully

23.3 Line-Following Testing

Sensor Pattern	Expected Movement	Observed Movement
010	Move straight	Robot moved forward
110	Soft left	Robot corrected left
100	Hard left	Robot turned left sharply
011	Soft right	Robot corrected right
001	Hard right	Robot turned right sharply
101	Move forward through center gap	Robot continued forward
000	Search slowly / stop after timeout	Robot searched and stopped after timeout

23.4 Mode Change Testing

Test Case	Expected Result	Observed Result
Robot starts	Robot begins in normal mode	Normal line following started
First cross-line detected	Robot changes to slow	Robot slowed down
Robot in slow mode with no IR signal	Robot continues slowly	Robot remained in slow mode
Exit cross-line detected without IR signal	Robot returns to normal mode	Robot speed increased again
IR signal received in slow mode	Robot enters parking mode	Parking sequence started

23.5 Parking Testing

Step	Expected Result	Observed Result
IR signal detected in slow mode	Parking mode activates	Parking sequence started

Forward movement	Robot moves slightly forward	Robot moved forward
Pivot left	Robot turns toward parking bay	Robot pivoted left
Reverse movement	Robot reverses into parking bay	Robot reversed into bay
Final stop	Robot stops permanently	Robot remained stopped

The testing results show that the robot successfully follows the line, detects cross-line markers, changes modes, receives the IR parking signal only during slow mode, performs parking, and stops safely when required.

24. Discussion

The completed robot satisfies the main requirements of the assignment. It can follow a black line using three IR sensors and make movement corrections based on the sensor pattern. The robot can detect perpendicular cross-line markers and use them to change between normal mode and slow parking-detection mode.

One of the most important parts of the project is the mode-changing logic. The robot does not simply follow a line continuously. It makes decisions based on cross-line detection and IR signal confirmation. This shows that the system has autonomous decision-making ability.

Another important feature is that the IR receiver is only checked during slow mode. This prevents incorrect parking outside the parking area. The robot must first detect the cross-line marker and enter the parking detection zone before the IR signal can trigger parking.

The use of counters and flags improves the reliability of the system. The cross-line confirmation counter prevents noise from being counted as a marker. The marker latch flag prevents the same marker from being counted multiple times. The IR confirmation counter prevents false parking due to a single unstable IR pulse. The line-lost counter improves safety by stopping the robot if it leaves the track.

The main limitation is that the robot uses N20 motors without encoders. Because of this, the robot cannot measure exact distance or wheel rotation. Therefore, the parking sequence is based on timing. This works well if the robot starts parking from a known position, but the final parking accuracy may change slightly depending on battery voltage, surface friction, and motor speed variation.

Overall, the project successfully demonstrates embedded system control, sensor-based navigation, motor control, state-machine design, and low-level Assembly programming.

25. Limitations

Although the robot meets the required functionality, there are some limitations.

First, the N20 motors do not have encoders. Therefore, the robot cannot measure exact wheel rotation, distance, or speed. The parking movement is based on fixed time delays, which may not always produce the exact same final position.

Second, the TCRT5000 sensors can be affected by lighting conditions, sensor height, and surface reflectivity. If the sensors are too high or too low, the readings may become unstable. Therefore, the sensor height and sensitivity must be adjusted carefully before testing.

Third, the parking movement is fixed. The robot does not use distance sensors or feedback control to adjust its parking position. Because of this, the parking sequence works best when the robot starts parking from a consistent position.

Fourth, the robot detects IR activity instead of decoding a specific remote-control command. This makes the system simple, but it may be affected by other IR sources if they are present inside the parking detection area.

Finally, because the system uses open-loop control, changes in battery voltage or surface friction can slightly affect motor speed and movement distance.

26. Conclusion

This project successfully developed an autonomous line-following and parking robot using an Arduino Uno, ATmega328P microcontroller, three TCRT5000 IR sensors, a VS1838 IR receiver, a TB6612 motor driver, and two N20 DC geared motors.

The robot is able to follow a black line, detect perpendicular cross-line markers, change operating modes, slow down near the parking area, detect a valid IR parking signal, and perform an automatic parking sequence. The system uses a finite state machine with three main modes: normal line-following mode, slow parking-detection mode, and parking execution mode. The cross-line counter controls the transition between normal and slow modes. The IR receiver confirms the selected parking bay only when the robot is in slow mode. This prevents incorrect parking outside the parking area. After receiving a valid IR signal, the robot performs a timed parking sequence and stops permanently.

The project also includes important safety and reliability features such as cross-line confirmation, duplicate marker prevention, IR signal validation, and line-lost timeout stopping. These features make the robot more stable and reliable during operation.

The firmware is implemented using AVR Assembly language, which allows direct register-level control of the microcontroller. This demonstrates low-level embedded system programming concepts such as port configuration, input reading, output control, PWM generation, timer usage, counters, flags, and program branching.

Overall, the project meets the assignment requirements and demonstrates a strong understanding of embedded systems, autonomous robotics, sensor interfacing, motor control, and Assembly-level programming.

References

- [1] Arduino, “Arduino Uno Rev3,” *Arduino Documentation*. [Online]. Available: <https://docs.arduino.cc/hardware/uno-rev3>. [Accessed: Apr. 26, 2026].
- [2] Microchip Technology Inc., “ATmega328/P 8-bit AVR Microcontroller Datasheet,” *Microchip Technology Inc.* [Online]. Available: https://docs.arduino.cc/resources/datasheets/Atmel-42735-8-bit-AVR-Microcontroller-ATmega328-328P_Datasheet.pdf. [Accessed: Apr. 26, 2026].
- [3] Microchip Technology Inc., “ATmega328P Automotive 8-bit AVR Microcontroller Datasheet,” *Microchip Technology Inc.* [Online]. Available: https://ww1.microchip.com/downloads/en/DeviceDoc/Atmel-7810-Automotive-Microcontrollers-ATmega328P_Datasheet.pdf. [Accessed: Apr. 26, 2026].